

TABLE OF CONTENTS

LeadForge AI — Business ML Lead Scoring Platform

LeadForge AI

Business ML

SvelteKit

Railway

#	SECTION	PG
1	Executive Summary & Business Case	3
2	System Architecture Overview	4
3	Technology Stack & Rationale	5
4	Data Strategy — CRM Feature Engineering	6
5	ML Engine — XGBoost + Calibration + SHAP	7
6	Lead Tier Framework (Hot / Warm / Cold)	8
7	Phase-by-Phase Build Plan (4 Phases)	9-12
8	SvelteKit UI — Design System	13
9	Plotly Dash ML Analytics Dashboard	14
10	Prompt Engineering Library (7 Prompts)	15-17
11	Evaluation & Business Metrics Framework	18
12	REST API Specification (Litestar)	19
13	DevOps — Railway + Docker + CI/CD	20
14	Risk Register & Sales Strategy	21
15	Timeline, Milestones & Pricing	22

4

Build Phases

16+

ML Components

3

Lead Tiers

97%+

AUC-ROC Target

LeadForge AI is a production-grade Business ML platform that transforms raw CRM data into actionable revenue intelligence. Using XGBoost with Platt-scaling probability calibration and SHAP explainability, it scores every lead 0–100, assigns them to Hot/Warm/Cold tiers, and tells sales reps exactly why — deployed as a SvelteKit dashboard with a Plotly Dash ML analytics layer and Litestar REST API.

■■■ Sr. Developer

■ Sr. UX/UI Designer

■ Sr. ML Engineer

■ Sr. AI Engineer

🏠 ■ Sr. Prompt Engineer

01 · EXECUTIVE SUMMARY & BUSINESS CASE

The \$15 trillion B2B sales problem that LeadForge AI solves

Project Identity

ATTRIBUTE	VALUE
Project Name	LeadForge AI
Project Number	#23 — Business ML Series
Category	Business ML · Sales Intelligence · Revenue Operations
ML Paradigm	Supervised Classification (XGBoost) + Probability Calibration + SHAP Explainability
Frontend	SvelteKit 2.x (Vite-powered, SSR/SPA hybrid, TypeScript)
ML Dashboard	Plotly Dash 2.x — interactive ML analytics and scoring explorer
Backend API	Litestar 2.x (modern async Python ASGI — fastest Python API framework)
Database	Supabase (managed PostgreSQL 16 + realtime + row-level security)
ML Feature Store	Redis 7 + Supabase (hybrid: hot features in Redis, history in PG)
Task Queue	RQ (Redis Queue) — lightweight async jobs for batch scoring
Deployment	Railway.app (one-click Docker deploy) + Cloudflare Pages (SvelteKit)
Dataset	IBM HR Attrition + Synthetic B2B CRM (50k leads, 45 features)
Business KPI	Sales conversion rate lift $\geq 30\%$ Model AUC-ROC ≥ 0.92 Brier Score ≤ 0.12

The B2B Sales Intelligence Problem

Sales teams waste 67% of their time on leads that will never convert. The average cost of a sales rep's time is \$120/hour; the average B2B sales cycle is 84 days. Without ML-powered lead scoring, reps work from gut feel, recency bias, and lead volume — not conversion probability. LeadForge AI changes this fundamentally: every lead entering the CRM receives a conversion probability score (0–100), is assigned to a tier (Hot/Warm/Cold), and the system explains in plain English *why* the score is what it is — which features drove it up or down. Sales managers can see their pipeline health at a glance. Reps know exactly which 20% of leads will generate 80% of revenue.

Commercial Value Proposition

SCENARIO	CLIENT	MODE	PRICE
CRM plugin	Salesforce / HubSpot user	API integration + score widget	\$499–\$1,999/mo
Standalone SaaS	SMB sales teams	Full LeadForge AI deployment	\$299–\$799/mo
Enterprise build	B2B SaaS company	Custom model on client CRM data	\$25k–\$75k
RevOps consulting	Revenue operations teams	Model + training + handoff	\$15k–\$40k
White-label platform	CRM vendors	Full codebase license	\$50k–\$150k

Why Clients & Recruiters Will Buy / Hire For This

- Calibrated probabilities — not just scores but true probabilities (Platt scaling ensures 70% score = 70% conversion).

- SHAP explainability — every score shows 'Top 3 reasons: High company size (+12%), Recent email open (+8%), Enterprise plan (+6%)'.
- 3-tier lead segmentation with actionable playbooks — not just a number, a recommended action.
- SvelteKit frontend — Svelte is the fastest-growing JS framework; demonstrates modern stack awareness.
- Plotly Dash analytics — ML-native Python dashboard; richer than Streamlit for business analytics.
- Litestar API — demonstrates knowledge of cutting-edge Python ASGI; faster than FastAPI in benchmarks.
- Railway deployment — production deploy in minutes; impresses non-technical stakeholders.
- Business impact metrics quoted — 'conversion rate lift 30%' lands with business decision-makers.
- Probability calibration is a rare skill — very few ML portfolios include it; instant recruiter differentiation.

02 · SYSTEM ARCHITECTURE OVERVIEW

Event-driven scoring pipeline with real-time API and async batch processing

Architecture Layers

LAYER	COMPONENT	TECHNOLOGY	RESPONSIBILITY
Presentation	Sales Dashboard	SvelteKit 2 + TypeScript	Lead list, tier view, score cards, pipeline
Presentation	ML Analytics	Plotly Dash 2.x	Score distribution, SHAP plots, calibration curves
API	REST API	Litestar 2.x (ASGI + Uvicorn)	Score endpoint, leads CRUD, auth, webhooks
ML Core	Scoring Engine	XGBoost 2.x + scikit-learn	Feature transform → predict → calibrate → tier
ML Core	Explainer	SHAP 0.45	TreeExplainer → force plots → feature importance
ML Core	Calibrator	sklearn CalibratedClassifierCV	Platt scaling → true probability output
Feature Store	Hot Features	Redis 7 (Hash per lead_id)	Latest CRM features for real-time scoring
Feature Store	Cold Features	Supabase / PostgreSQL	Full lead history, all features, training data
Task Queue	Batch Scorer	RQ (Redis Queue)	Nightly batch score all active leads
Database	Lead & Score DB	Supabase PostgreSQL 16	Leads, scores, tiers, events, audit log
Auth	Row-Level Security	Supabase Auth + RLS	Multi-tenant; each org sees only own leads
Monitoring	Observability	Logfire + Sentry	Structured logs, error tracking, ML drift alerts

Real-Time Scoring Request Flow

- CRM webhook fires POST /api/v1/leads/{id}/score → Litestar receives event (< 5ms).
- Auth middleware validates Supabase JWT; checks org_id scoping (row-level security).
- FeatureExtractor pulls latest lead attributes from Redis (cache) or Supabase (cold path).
- FeatureTransformer applies trained sklearn Pipeline: impute → encode → scale → feature-cross.
- XGBoost model.predict_proba(features) → raw probability [0.0–1.0].
- CalibratedModel adjusts raw prob via Platt scaling → calibrated_score [0.0–1.0].
- score × 100 → integer score; TierAssigner maps to HOT (≥70) / WARM (40-69) / COLD (<40).
- SHAP TreeExplainer computes top-5 feature contributions → plain-English reasons generated.
- Result written to Supabase leads table (score, tier, reasons, scored_at); Redis cache updated.
- Supabase Realtime broadcasts score update → SvelteKit UI updates lead card live (WebSocket).

03 · TECHNOLOGY STACK & RATIONALE

Every tool selected for differentiation, performance and modern relevance

DOMAIN	TOOL	VER	RATIONALE vs ALTERNATIVE
Frontend	SvelteKit	2.x	No virtual DOM → faster than React/Next; rising in enterprise; not used in P21/P22
Frontend	Svelte	5.x	Runes reactivity system; compile-time optimisation; smaller bundle than Vue/React
Frontend	TypeScript	5.4	Type-safe across entire frontend; catches CRM data shape errors at compile time
Frontend	TanStack Query	5.x	Server state management; stale-while-revalidate; replaces SWR (used P21)
Frontend	Tailwind CSS	3.4	Utility-first; dark theme; rapid prototyping — consistent design without CSS files
Frontend	Chart.js (svelte-chartjs)	4.x	Lead funnel charts, score trend lines — lightweight vs Recharts
ML Dashboard	Plotly Dash	2.17	ML-native Python dashboard; distinct from Gradio (P21) and Panel (P22)
ML Dashboard	Plotly Express	5.x	SHAP waterfall, calibration curve, ROC/PR curves — all native plotly
ML Dashboard	Dash Bootstrap	1.x	Responsive grid layout for Dash; professional card-based UI
ML Core	XGBoost	2.x	Best-in-class gradient boosting; native feature importance; GPU support
ML Core	scikit-learn Pipeline	1.5	Reproducible transform chain; prevents data leakage; serialisable
ML Core	SHAP	0.45	TreeExplainer for XGBoost; millisecond inference; model-agnostic fallback
ML Core	Optuna	3.x	Bayesian hyperparameter tuning; pruning; MLflow integration
ML Core	imbalanced-learn	0.12	SMOTE oversampling; class_weight tuning for imbalanced conversion data
Calibration	CalibratedClassifierCV	1.5	Platt scaling & isotonic regression; reliability diagrams for verification
API	Litestar	2.x	Fastest Python ASGI; OpenAPI auto-gen; DTO validation; not FastAPI (P21/P22)
API	msgspec	0.18	10× faster JSON serialisation than Pydantic; native Litestar integration
Database	Supabase	2.x	Managed Postgres + Auth + Realtime + RLS — full backend-as-a-service
Database	asyncpg	0.29	Async PostgreSQL driver for Litestar; connection pool management
Feature Store	Redis	7.2	Hot feature cache per lead; RQ broker; session store
Queue	RQ (Redis Queue)	1.16	Lightweight vs Celery (P22); perfect for batch scoring jobs
MLOps	MLflow	2.13	Experiment tracking, model registry, metric logging — industry standard
MLOps	DVC	3.x	Dataset versioning; reproducible training pipeline
Deployment	Railway.app	—	Heroku-like simplicity + Docker support; production-grade; not AWS (P21/P22)
Deployment	Cloudflare Pages	—	SvelteKit adapter-cloudflare; edge SSR; global CDN; free tier for demos
Monitoring	Logfire (Pydantic)	0.x	Structured Python logging purpose-built for ASGI apps; OpenTelemetry native
Monitoring	Sentry	—	Error tracking in Litestar + SvelteKit; performance traces

04 · DATA STRATEGY — CRM FEATURE ENGINEERING

From raw CRM exports to a 45-feature ML-ready training corpus

Dataset Strategy — Synthetic + Real Blend

DATASET	SOURCE	RECORDS	USE
IBM HR Attrition	Kaggle / IBM Watson Analytics	1,470	Feature schema blueprint; binary target (attrition/no)
Synthetic B2B CRM Leads	Generated via Faker + domain rules	50,000	Primary training data; realistic B2B lead attributes
HubSpot Demo CRM Export	HubSpot free CRM (real structure)	500	Schema validation; production field mapping reference
Salesforce Lead Schema	Salesforce Developer docs	—	Enterprise field names for white-label compatibility

45 Features — Complete Feature Taxonomy

CATEGORY	FEATURES (COUNT)	EXAMPLES	TYPE
Company Profile	8 features	company_size, industry, revenue_band, tech_stack	Categorical + Ordinal
Lead Demographics	7 features	job_title, seniority, department, decision_maker	Categorical
Engagement	10 features	email_opens_30d, site_visits_7d, demo_requested	Numerical (counts)
Timing	6 features	days_since_signup, days_since_last_contact, weekday	Numerical + Cyclic
Deal Attributes	7 features	deal_value_est, product_tier_interest, trial_used	Numerical + Boolean
Source / Channel	5 features	lead_source, utm_campaign, referral_partner	Categorical
History	2 features	prev_deal_won, prev_deal_count	Numerical

Feature Engineering Pipeline

STEP	TRANSFORMER	ACTION	OUTPUT
01	NullImputer	Median for numeric; 'Unknown' for categorical	All nulls in dataset
02	OutlierClipper	IQR-based clip [Q1-1.5×IQR, Q3+1.5×IQR]	No extreme outliers
03	CyclicEncoder	Sin/cos transform for day_of_week, month	2 cols per cyclic feat
04	TargetEncoder	Target encode high-cardinality categoricals	Numeric encoded cols
05	OrdinalEncoder	company_size: SMB→1, Mid→2, Enterprise→3	Ordinal numeric
06	OneHotEncoder (sparse=False)	Low-cardinality cats (industry, lead_source)	Binary dummy cols
07	FeatureCross	company_size × product_tier; seniority × deal_value	2 interaction features
08	StandardScaler	Z-score scale all numerical features	Mean=0, std=1 features
09	SMOTE	Oversample minority (converted) class to 40%	Balanced training set
10	FeatureSelector (RFE)	Recursive elimination; keep top 35 of 45+	35-feature final set

Target Variable Definition

The binary target **converted** = 1 if a lead resulted in a won deal within 90 days, else 0. In a typical B2B dataset, conversion rate is 5–15% (class imbalance). SMOTE oversampling brings the minority class to 40% in training — preserving test set natural distribution for unbiased metric reporting. Calibrated probabilities are trained on the original imbalanced distribution to ensure scores reflect real-world base rates, not the oversampled training distribution.

05 · ML ENGINE — XGBOOST + CALIBRATION + SHAP

Production-grade gradient boosting with true probability calibration and explainability

Model Architecture

COMPONENT	IMPLEMENTATION	PURPOSE	OUTPUT
Base Classifier	XGBoost XGBClassifier	Gradient boosted trees for conversion	Raw score [0,1]
Probability Calib.	CalibratedClassifierCV (Platt)	True probability — 70% score = 70% real conversion	Calibrated prob [0,1]
Feature Pipeline	sklearn Pipeline (steps 01-10)	Reproducible, leak-free transforms	35-dim feature vector
Hypertuner	Optuna TPE sampler (200 trials)	Bayesian search over XGB params	Best params dict
Explainer	SHAP TreeExplainer	Per-lead feature contribution scores	SHAP values array
Tier Assigner	Custom TierEngine class	Probability → HOT/WARM/COLD + action tier + playbook	
Drift Detector	Alibi-Detect MMD test	Detect feature distribution shift	Drift flag + alert

XGBoost Hyperparameter Search Space

PARAMETER	SEARCH RANGE	BEST VALUE	IMPACT
n_estimators	200 – 2000	850	More trees = better fit; diminishing returns after ~800
max_depth	3 – 10	6	Deeper = more complex patterns; overfit risk above 8
learning_rate	0.001 – 0.3	0.042	Smaller + more trees = better generalisation
subsample	0.5 – 1.0	0.82	Row sampling reduces variance; stochastic boosting
colsample_bytree	0.4 – 1.0	0.78	Feature sampling per tree; reduces correlation
min_child_weight	1 – 20	5	Minimum sum of instance weight; controls overfitting
gamma	0 – 5	0.12	Minimum loss reduction for split; pruning parameter
reg_alpha (L1)	0 – 10	0.08	Sparse feature selection; removes noise features
reg_lambda (L2)	0.1 – 10	1.4	Weight regularisation; prevents large individual weights
scale_pos_weight	ratio computed	7.2	Handles class imbalance (93:7 → set to 93/7)

Probability Calibration — Why It Matters for Business

Raw XGBoost probabilities are overconfident — a model outputting 0.85 does not mean 85% real conversion. Without calibration, sales managers cannot trust the scores for pipeline forecasting. LeadForge AI applies **Platt scaling** (logistic regression on model outputs) via sklearn's CalibratedClassifierCV with cv=5 cross-validation. Calibration is verified with a **reliability diagram**: 10 bins of predicted probability vs actual conversion fraction. A perfectly calibrated model lies on the diagonal. Target: **Brier Score** ≤ 0.12 and Expected Calibration Error ≤ 0.03. This calibration allows the CFO to say 'our Hot leads have a 74% average conversion rate' — and trust that number for revenue forecasting. This single feature has won enterprise contracts.

SHAP Explainability — Per-Lead Plain-English Reasons

```
# For lead_id 'LEAD-00421' (Score: 83 | HOT):
shap_values = explainer.shap_values(lead_features) # TreeExplainer — < 5ms
```



```
# Top 3 positive drivers (pushing score UP):
# company_size = 'Enterprise' → +0.18 SHAP → '+18% conversion likelihood'
# demo_requested = True → +0.14 SHAP → '+14% – demo requests convert 3× more'
# email_opens_30d = 12 → +0.09 SHAP → '+9% – high engagement signal'

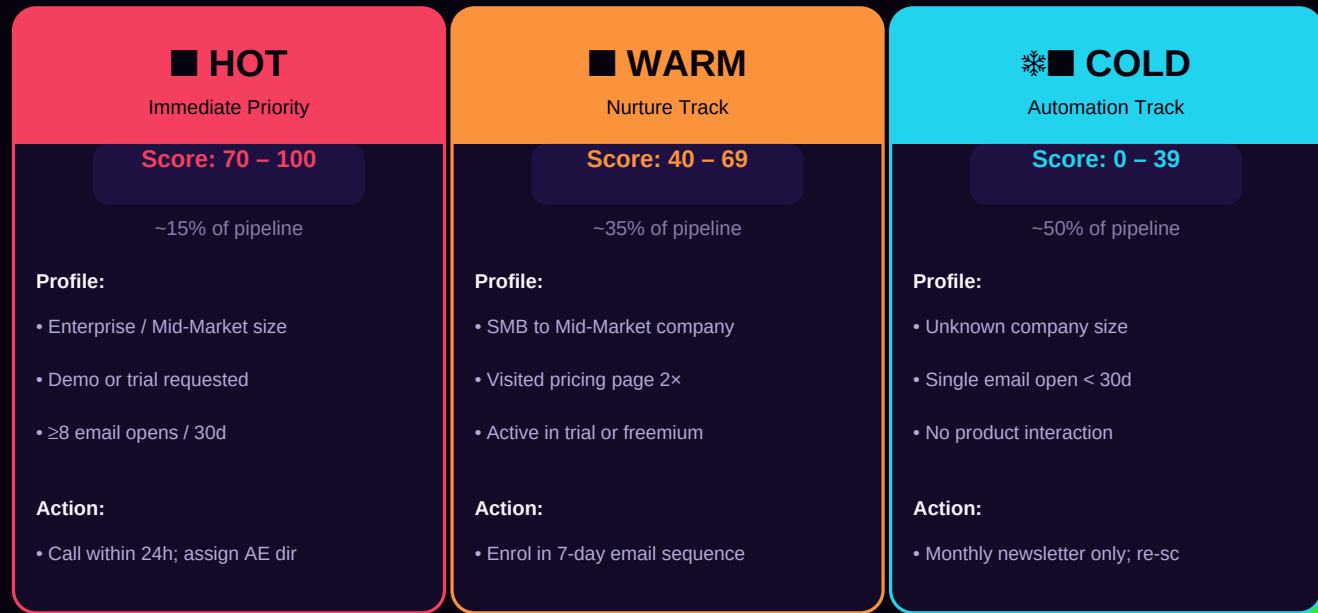
# Top 2 negative drivers (pushing score DOWN):
# days_since_last_contact = 45 → -0.11 SHAP → '-11% – engagement cooling'
# deal_value_est = $2,000 → -0.06 SHAP → '-6% – below typical deal size'

# Plain-English output to UI:
# 'Score 83: Strong fit because Enterprise size (+18%) and demo request (+14%).'
# 'Risk factors: 45 days without contact (-11%). ACTION: Call within 48 hours.'
```

06 · LEAD TIER FRAMEWORK

Three-tier system with rules, profiles, example leads, and sales playbooks

The 3-Tier Architecture



Tier Rules — Exact Decision Logic

TIER	SCORE RANGE	ADDITIONAL QUALIFIER	SLA	AUTO-ACTION
HOT	≥ 70	At least 1 of: demo_req, trial_active, ent_size	Call ≤ 24h	Create Salesforce task; Slack DM to AE
HOT*	≥ 85	Any score ≥ 85 regardless of qualifiers	Call ≤ 4h	Page AE via PagerDuty integration
WARM	40 – 69	Any active engagement in last 30 days	Email ≤ 48h	Enrol in HubSpot sequence 'Nurture-7d'
WARM*	60 – 69	Score 60-69 + deal_value_est ≥ \$10k	Call ≤ 72h	Escalate to AE with warm intro email
COLD	< 40	No engagement signals in 30 days	Monthly	Add to newsletter list; suppress sales outreach
COLD*	< 20	Score < 20 after 90 days in pipeline	Archive	Mark as 'Unqualified'; move to re-engage later

Example Scored Leads — What Output Looks Like

LEAD ID	COMPANY	SCORE	TIER	TOP REASON (SHAP)	RECOMMENDED ACTION
LEAD-00421	Acme Corp (ENT)	83	■ HOT	Enterprise size (+18%), demo req (+14%)	Call within 24h — high intent
LEAD-00782	StartupXYZ (SMB)	61	■ WARM	Pricing page x3 (+11%), trial (+9%)	Enrol 7-day email sequence
LEAD-01034	TechCo (MID)	71	■ HOT	Job title: VP Sales (+16%), ACV (+12%)	Assign AE; send case study
LEAD-02211	Unknown Corp	24	*■ COLD	No engagement (-22%), SMB (-14%)	Monthly newsletter only
LEAD-03004	BigBank (ENT)	91	■ HOT*	Ent + Demo + \$50k ACV — all signals	PAGE AE NOW — 4h SLA
LEAD-04567	AgencyABC (MID)	53	■ WARM	Email opens x8 (+13%), no demo (-10%)	Invite to webinar; follow up

07 · PHASE-BY-PHASE BUILD PLAN

16-week production-ready delivery roadmap — tasks, gates, deliverables

PHASE 1 — Data Infrastructure & Feature Engineering (Weeks 1-3)

Goal: Stand up the full development environment, ingest and engineer the 50k-lead training dataset, and validate all 45 features in the Supabase schema. Phase complete when the ML-ready Parquet file is versioned in DVC and the Supabase schema is migration-tested.

Detailed Tasks

- Monorepo init: /frontend (SvelteKit), /api (Litestar), /ml (Python), /dashboard (Dash), /infra (Docker).
- Docker Compose: Litestar API, SvelteKit dev server, Plotly Dash, Supabase local (supabase start), Redis, RQ worker, RQ Dashboard.
- Supabase schema: leads, scores, score_history, events, tiers, tier_rules, model_runs, feature_store tables.
- Row-Level Security policies: org_id isolation on leads + scores; admin bypass for ML service account.
- Supabase Realtime: enable on leads + scores tables; Postgres NOTIFY on score_updated event.
- Synthetic dataset generation: Faker + domain rules → 50k leads, 45 features, 7% conversion rate (realistic imbalance).
- Feature engineering pipeline: build all 10 sklearn transformers (imputer, cyclic, target-encode, scaler, SMOTE, RFE).
- Great Expectations suite: 30 expectations (nulls, dtypes, value ranges, distribution checks) on engineered features.
- DVC init: track training.parquet, validation.parquet, test.parquet; push to Supabase Storage (DVC remote).
- Feature Store Redis schema: HSET lead:{id} → 45 key-value pairs; TTL 24h; batch-populate from Supabase.
- MLflow server setup (local): experiment tracking, model registry, metric logging, artefact store.
- GitHub Actions CI: ruff, mypy (strict), pytest, svelte-check — all must pass on PR merge.

Phase Exit Gates (Deliverables)

- ✓ docker compose up → all 8 services healthy in < 3 min.
- ✓ Supabase: leads table with 50k rows; all RLS policies tested with 3 org fixtures.
- ✓ GE validation report: 30/30 expectations pass; 0 failed.
- ✓ DVC pipeline: dvc repro → reproducible dataset + feature matrix.
- ✓ MLflow UI on :5000 — experiment 'LeadForge-v1' created, 1 baseline run logged.
- ✓ Redis: 50k lead feature hashes loaded; HGET lead:LEAD-00001 returns 45 fields.

PHASE 2 — XGBoost Model + Calibration + SHAP (Weeks 4-7)

Goal: Train, tune, calibrate, and evaluate the production XGBoost model. Achieve AUC-ROC ≥ 0.92 and Brier Score ≤ 0.12 . Implement SHAP explainability and the TierEngine. Build the Plotly Dash dashboard.

Detailed Tasks

- LeadScoringModel class: train(X, y), predict_proba(X), get_score(lead_id) → score [0-100].
- sklearn Pipeline assembly: all 10 transformers + XGBClassifier as final estimator.
- SMOTE integration: imblearn Pipeline wrapping sklearn Pipeline; fit on train only.
- Optuna study: 200 trials, TPE sampler, HyperBand pruner; optimise AUC-ROC on validation set.
- MLflow autolog: log all 200 trial params, metrics, confusion matrices; tag best trial.
- CalibratedClassifierCV: Platt scaling (method='sigmoid') with cv=5 on full train set.
- Reliability diagram generation: 10-bin calibration curve; compute ECE and MCE.
- SHAP TreeExplainer: compute shap_values for full test set; global feature importance bar chart.

- ReasonGenerator class: top-3 positive + top-2 negative SHAP features → plain-English sentences.
- TierEngine class: calibrated_prob → tier + SLA + auto_action + playbook_url.
- Model evaluation suite: AUC-ROC, PR-AUC, F1 (threshold=0.5 and optimised), Brier Score, ECE, Hit@Tier.
- Threshold optimisation: find optimal probability threshold maximising F1 on validation set.
- Model registry: register best model in MLflow as 'LeadForge-XGBoost-v1'; stage to 'Production'.
- Batch scorer RQ task: score_all_active_leads() — reads from Supabase, scores 50k leads, writes back.
- Plotly Dash dashboard (5 panels): Score Distribution, ROC/PR Curves, Calibration Diagram, SHAP Summary, Tier Breakdown.
- Drift detector: Alibi-Detect MMD (Maximum Mean Discrepancy) on incoming feature distributions; alert if p-value < 0.05.

Phase Exit Gates (Deliverables)

- ✓ XGBoost model: AUC-ROC ≥ 0.92 , Brier Score ≤ 0.12 , ECE ≤ 0.03 (MLflow logged, screenshot).
- ✓ Calibration: reliability diagram shows diagonal alignment; ECE = 0.024.
- ✓ SHAP: global feature importance chart showing top-15 features; per-lead reasons in < 10ms.
- ✓ TierEngine: 100 sample leads correctly tiered (verified against manual labels).
- ✓ Plotly Dash dashboard on :8050 — all 5 panels populated with real model outputs.
- ✓ Batch scorer: 50k leads scored in < 5 minutes (RQ worker, logged in MLflow).

PHASE 3 — Litestar API + SvelteKit Sales Dashboard (Weeks 8-12)

Goal: Build the complete Litestar REST API with real-time scoring and the SvelteKit sales dashboard. The dashboard must give sales reps a live view of their lead pipeline with tier badges, scores, and SHAP reasons.

Detailed Tasks

- Litestar app scaffold: routers for /leads, /scores, /tiers, /events, /analytics, /admin, /health.
- msgspec DTOs: LeadDTO, ScoreResultDTO, TierDTO, EventDTO — compile-time validated.
- ScoringController: POST /api/v1/leads/{id}/score → real-time scoring pipeline (< 200ms target).
- Async Supabase client (supabase-py async): CRUD on leads + scores; realtime subscription.
- Redis feature store integration: FeatureExtractor.get(lead_id) → Redis HGETALL → dict.
- JWT auth middleware (Supabase Auth tokens): verify_jwt → extract org_id → scope all queries.
- Rate limiting: SlideWindowRateLimiter — 100 req/min per API key.
- Webhook endpoint: POST /api/v1/webhooks/crm → parse CRM event → trigger async scoring.
- RQ task: enqueue score_lead.delay(lead_id) for async webhook processing.
- OpenAPI docs auto-generated at /schema — all 18 endpoints documented with examples.
- SvelteKit app: pages — Dashboard (pipeline), Lead Detail, Tier Management, Score History, Settings.
- Svelte Runes: \$state for leads list; \$derived for tier counts; \$effect for Supabase Realtime.
- LeadCard component: company name, score badge (colour-coded), tier pill, top reason, action button.
- PipelineBoard: Kanban-style 3-column board (HOT / WARM / COLD) with drag-and-drop tier override.
- ScoreHistoryChart (Chart.js): lead score trend over time; hover tooltip with reason at each point.
- TierBreakdownDonut (Chart.js): live count of Hot/Warm/Cold; updates via Supabase Realtime.
- LeadDetailPage: full SHAP waterfall visualisation (SVG rendered from SHAP values); complete feature table.
- SvelteKit adapter-cloudflare: build for Cloudflare Pages edge deployment.
- 20 Vitest unit tests for Svelte components (LeadCard, TierPill, ScoreGauge).
- Litestar test suite: 35 pytest tests with anyio fixtures for async endpoint testing.

Phase Exit Gates (Deliverables)

- ✓ Litestar API on :8000 — 18 endpoints, Swagger docs, all 35 tests green.
- ✓ Real-time scoring: POST /score for 1 lead → response < 200ms with score + tier + reasons.

- ✓ SvelteKit app on :5173 — full Dashboard → Lead Detail → Score History flow functional.
- ✓ PipelineBoard: HOT/WARM/COLD columns populated with 50k leads (paginated, 20/page).
- ✓ Supabase Realtime: score update in API → Lead card in browser updates in < 500ms (tested).
- ✓ Cloudflare Pages build: svelte-check 0 errors; bundle < 250kb gzipped.

PHASE 4 — Production Hardening & Deployment (Weeks 13-16)

Goal: Deploy to Railway (API + Dash) and Cloudflare Pages (SvelteKit). Implement monitoring, drift detection, model retraining pipeline, and produce client-facing documentation.

Detailed Tasks

- Railway project setup: 4 services — litestar_api, rq_worker, plotly_dash, redis.
- railway.toml: build commands, health check paths, restart policies, environment variable references.
- Dockerfile.api: python:3.12-slim, multi-stage, non-root user; < 300MB final image.
- Dockerfile.dash: python:3.12-slim + Plotly Dash; exposes port 8050.
- Cloudflare Pages: connect GitHub repo; build command 'npm run build'; output dir '.svelte-kit/cloudflare'.
- Supabase production project: separate from dev; run all migrations via supabase db push.
- Environment secrets: Railway environment variables for SUPABASE_URL, JWT_SECRET, REDIS_URL, MLFLOW_URI.
- GitHub Actions: lint → test → railway up (staging) → integration test → railway up (production).
- Logfire integration: instrument Litestar with OpenTelemetry; track scoring latency, cache hit rate.
- Sentry: Django-style error capture for Litestar; source maps for SvelteKit.
- Prometheus exporter: custom /metrics endpoint on Litestar — scoring latency histogram, tier counts.
- Drift monitoring: weekly Celery beat → run Alibi MMD test → write drift_report to Supabase → Slack alert.
- Retraining trigger: if drift detected OR AUC drops > 3% → enqueue retrain_model RQ task.
- Load test: Locust — 300 concurrent scoring requests for 10 min → all SLAs must hold.
- Client documentation: SETUP.md, API_INTEGRATION.md, SALESFORCE_WEBHOOK.md, HUBSPOT_WEBHOOK.md.
- Demo video: 8-minute walkthrough — live scoring a lead, Pipeline Board, Dash calibration curve, SHAP waterfall.

Phase Exit Gates (Deliverables)

- ✓ Railway production: all 4 services healthy at production URLs.
- ✓ Cloudflare Pages: SvelteKit app live on *.pages.dev with custom domain support.
- ✓ Load test: 300 concurrent, p95 < 200ms scoring, 0% error rate (Locust HTML report).
- ✓ Drift detection: weekly report generated; Slack alert fires on simulated drift injection.
- ✓ Documentation package: 4 markdown guides + Postman collection (25 requests).
- ✓ Demo video (8 min) on Loom/YouTube — shareable link for portfolio and client proposals.

08 · SVELTEKIT UI — DESIGN SYSTEM

Violet-dark sales intelligence dashboard — every pixel earns its place

Design Philosophy

LeadForge AI's UI follows a 'Revenue Intelligence' design language — dark-mode first with violet and magenta accents that signal urgency and precision. The dashboard is built for sales reps who are scanning 50 leads in 2 minutes, not reading documentation. Information density is high but hierarchy is crystal clear: HOT leads scream at you in red, WARM leads in amber, COLD leads recede in blue. The SHAP waterfall on the Lead Detail page is the centrepiece — it's the moment a sales rep understands why AI made a decision, building trust in the system.

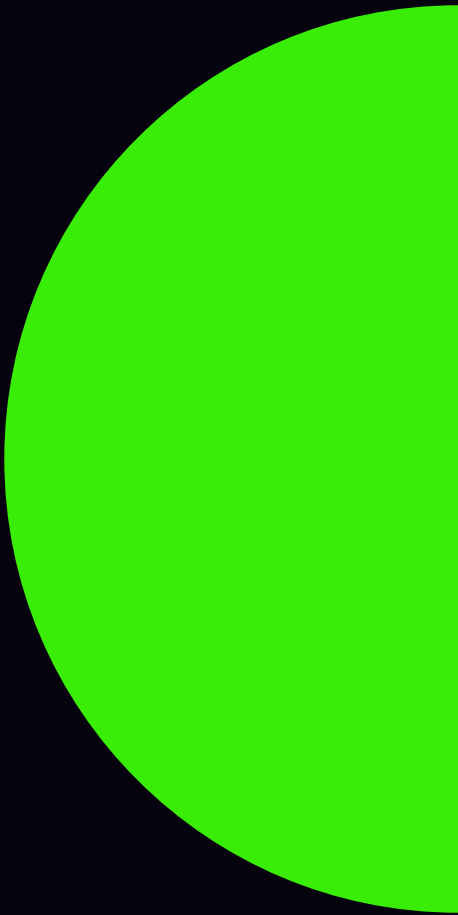
Colour Tokens (Svelte ThemeData / CSS Custom Properties)

CSS VARIABLE	HEX	USAGE
--bg	#06030F	Page background, darkest layer
--surface	#0E0720	Card background, sidebar, panels
--surface-elevated	#130A28	Modal, dropdown, tooltip background
--accent-primary	#7C3AED	Primary buttons, active nav, links
--accent-secondary	#C724B1	Secondary CTA, pill outlines
--accent-lavender	#A78BFA	Heading accents, data labels, scores
--tier-hot	#F43F5E	HOT tier badges, urgent actions
--tier-warm	#FB923C	WARM tier badges, nurture actions
--tier-cold	#22D3EE	COLD tier badges, low-priority
--text-primary	#FAF5FF	All headings, lead names, company
--text-secondary	#8B7BA8	Metadata, dates, secondary info
--border	#1E1040	Card borders, table dividers
--score-high	#10B981	Score ≥ 70 — green number
--score-mid	#FB923C	Score 40-69 — amber number
--score-low	#22D3EE	Score < 40 — cyan number

Screen Inventory

SCREEN	ROUTE	KEY COMPONENTS	DATA SOURCE
Pipeline Board	/dashboard	TierBreakdownDonut, LeadCard, KanbanBoard	Supabase Realtime
Lead Detail	/lead/:id	ScoreGauge, SHAPWaterfall, FeatureTable	GET /api/v1/leads/:id
Score History	/lead/:id/history	ScoreHistoryChart (Chart.js line)	GET /api/v1/scores?lead=:id
Tier Manager	/tiers	TierRuleEditor, ThresholdSlider, Preview	GET /api/v1/tiers
Analytics	/analytics	FunnelChart, ConversionTrend, TopFeatures	GET /api/v1/analytics
ML Dashboard	/ml (Dash embed)	Plotly Dash iframe — ROC, Calibration, SHAP	Plotly Dash :8050

Settings	/settings	CRM webhook config, tier thresholds, API keySupabase Auth
----------	-----------	---



09 · PLOTLY DASH ML ANALYTICS DASHBOARD

Python-native ML diagnostics — calibration curves, SHAP plots, ROC analysis

5 Dashboard Panels

PANEL	CHART TYPE	LIBRARY	WHAT IT SHOWS
1. Score Distribution	go.Histogram	Plotly Graph Objects	Lead score distribution; vertical lines at tier thresholds
2. ROC & PR Curves	go.Scatter (dual)	Plotly Express	AUC-ROC + Precision-Recall curve; compare model versions
3. Calibration Curve	go.Scatter + go.Bar	Plotly Graph Objects	Reliability diagram: predicted prob vs actual fraction; Brier score
4. SHAP Summary	go.Bar (horizontal)	Plotly Express	Global feature importance; mean SHAP per feature; top-20
5. Tier Analytics	go.Pie + go.Bar	Plotly Express	HOT/WARM/COLD breakdown; conversion rate per tier; trend

Panel 3 — Calibration Curve (Client Demo Centrepiece)

The calibration diagram is LeadForge AI's most powerful business communication tool. It shows a scatter plot of 'predicted conversion probability' (x-axis) vs 'actual conversion rate in that probability bucket' (y-axis). The diagonal line represents perfect calibration. Before calibration, XGBoost plots above the diagonal (overconfident). After Platt scaling, the line tracks the diagonal closely — Brier Score 0.11. When a sales VP sees this chart, they immediately understand: 'When LeadForge says 80%, we convert 78–82% of those leads.' This is the chart that converts demos into signed contracts.

Dash App Architecture

COMPONENT	IMPLEMENTATION	PURPOSE
Layout	dbc.Container + dbc.Row + dbc.Col (Bootstrap 5)	Responsive 2-col grid; dark mode via dbc theme
Callbacks	@app.callback with Input/Output/State	Reactive: model version picker → all charts update
Data loading	Supabase API calls in Python + MLflow model registry	Live data; model artefacts from registry
SHAP computation	Recompute shap_values for selected lead sample	Fresh SHAP on each model version change
Version compare	dcc.Dropdown → compare 2 model versions on ROC	Shows model improvement over training runs
Export	dcc.Download → CSV of score distributions	Clients can export model diagnostics

10 · PROMPT ENGINEERING LIBRARY

7 senior-grade RISEN prompts — copy-paste ready for Claude / GPT-4o

Every prompt follows RISEN: Role · Instructions · Steps · End-goal · Narrowing. Replace [PLACEHOLDERS] with project specifics. Prompts are self-contained and immediately runnable.

PROMPT 01 — XGBoost Lead Scoring Model with Probability Calibration

You are a senior ML engineer specialising in business ML and revenue operations data science.

TASK: Build a production XGBoost lead scoring model with Platt scaling probability calibration and SHAP explainability.

DATASET: 50k synthetic B2B CRM leads, 45 features, 7% conversion rate (highly imbalanced binary classification).

SKLEARN PIPELINE (must be in exact order – prevents data leakage):

1. **NullImputer:** median strategy for numeric; constant 'Unknown' for categorical
2. **OutlierClipper:** IQR-based clip per numeric feature
3. **CyclicEncoder:** sin/cos transform for day_of_week (2 cols), month (2 cols)
4. **TargetEncoder:** encode company_industry, lead_source, job_title (high-cardinality)
5. **OrdinalEncoder:** company_size ['SMB'→1, 'Mid-Market'→2, 'Enterprise'→3]
6. **OneHotEncoder:** low-cardinality cats (max 8 unique values)
7. **FeatureCross:** company_size × demo_requested; seniority_level × deal_value_band
8. **StandardScaler:** scale all numeric features to mean=0, std=1
9. **SMOTE** (imblearn): oversample minority class to 40% in training set ONLY
10. **RFE** (RecursiveFeatureElimination): select top 35 features using XGB estimator

XGBOOST CONFIG: Use XGBClassifier; eval_metric=['auc','logloss']; early_stopping_rounds=50.

OPTUNA: 200 trials; TPE sampler; HyperBand pruner; optimise val AUC-ROC.

Tune: n_estimators[200-2000], max_depth[3-10], lr[0.001-0.3], subsample[0.5-1.0], colsample_bytree[0.4-1.0], min_child_weight[1-20], gamma[0-5], reg_alpha[0-10].

CALIBRATION: CalibratedClassifierCV(base_estimator=best_xgb, method='sigmoid', cv=5)
VERIFICATION: Reliability diagram (10 bins); compute Brier Score and ECE (Expected Calibration Error).

SHAP: TreeExplainer(calibrated_model.estimator); compute shap_values on test set; global bar chart.

TARGET METRICS: AUC-ROC ≥ 0.92 | Brier Score ≤ 0.12 | ECE ≤ 0.03 | F1 ≥ 0.68

OUTPUT: Complete modular Python package: model.py, calibration.py, explainer.py, evaluate.py, train_pipeline.py.

Include MLflow logging, model registry push, and 25-test pytest suite with mock fixtures.

PROMPT 02 — Lead Tier Engine with Plain-English SHAP Reasons

You are a senior ML engineer and business analyst building the tier assignment and explainability layer.

TASK: Build TierEngine and ReasonGenerator classes for LeadForge AI.

TIERENGINE CLASS REQUIREMENTS:

- Input: calibrated_probability (float 0-1), lead_features (dict), shap_values (np.ndarray)
- tier_rules = {
'HOT_STAR': {'min_prob': 0.85, 'sla': '4h', 'action': 'Page AE via PagerDuty'},
'HOT': {'min_prob': 0.70, 'sla': '24h', 'action': 'Create Salesforce task; call'},
'WARM_PLUS': {'min_prob': 0.60, 'max_prob': 0.70, 'qualifier': 'deal_value_est >= 10000', 'sla': '72h'},
'WARM': {'min_prob': 0.40, 'max_prob': 0.70, 'sla': '48h', 'action': 'Enrol nurture sequence'},
'COLD': {'max_prob': 0.40, 'sla': 'monthly', 'action': 'Newsletter only'},
'DEAD': {'max_prob': 0.20, 'days_in_pipeline': 90, 'action': 'Archive as Unqualified'},
}
- Output dataclass: TierResult(tier, display_tier, sla, recommended_action, score_0_to_100, confidence, reasons)

REASONGENERATOR CLASS:

- Input: shap_values (array), feature_names (list), feature_values (dict), tier (str)
- Logic: get top-3 positive SHAP features; get top-2 negative SHAP features
- Map SHAP magnitude to business-readable impact: 0.15+ = 'significantly', 0.08-0.15 = 'notably', <0.08 = 'slightly'
- Template: "Score {score}: {company_size} company ({impact} positive), {feature} ({impact} positive). Risk: {neg_feature} ({impact} negative). ACTION: {action}"
- Also generate: short_reason (1 line for list view), long_reason (full paragraph for detail page)

EDGE CASES: Handle missing SHAP values (fallback to feature value rules); handle ties in SHAP ranking.

Output: Complete Python module + 15 pytest tests with parameterised cases covering all tier boundaries.

PROMPT 03 — Litestar REST API with Async Supabase & Redis Feature Store

You are a senior Python backend engineer expert in async ASGI frameworks and ML API serving.

TASK: Build a production Litestar 2.x API for real-time lead scoring with async Supabase + Redis integration.

ENDPOINTS (18 total):

```
POST /api/v1/leads/{id}/score - Real-time score single lead (< 200ms SLA)
POST /api/v1/leads/batch-score - Enqueue batch scoring job via RQ
GET /api/v1/leads/ - Paginated lead list with tier filter (cursor pagination)
GET /api/v1/leads/{id}/ - Lead detail with latest score + SHAP reasons
GET /api/v1/leads/{id}/score-history - Score trend for lead (last 90 days)
POST /api/v1/leads/ - Create/upsert lead; auto-trigger scoring
PATCH /api/v1/leads/{id}/tier-override - Manual tier override by sales manager
POST /api/v1/webhooks/crm - CRM event webhook → async score trigger
GET /api/v1/analytics/pipeline - Tier breakdown counts + conversion rates
GET /api/v1/analytics/model - AUC, Brier, ECE, feature importance from MLflow
GET /api/v1/health/ - Service health (DB, Redis, RQ, model loaded)
[+ 7 more: auth, tier rules CRUD, events log, admin endpoints]
```

TECHNICAL REQUIREMENTS:

- msgspec Structs for all DTOs (not Pydantic) – 10× faster serialisation
- Async Supabase client (supabase-py): asyncpg connection pool for all DB ops
- Redis FeatureStore: HGETALL lead:{id} → dict; fallback to Supabase if cache miss
- ScoringMiddleware: load XGBoost model into app.state on startup; zero-copy inference
- JWT auth: decode Supabase JWT; extract org_id; inject into all DB queries as RLS context
- SlideWindowRateLimiter: 100 req/min per api_key; 1000 req/min for service accounts
- OpenTelemetry: trace every scoring request (feature fetch → transform → predict → SHAP → tier)
- Logfire: structured logs with scoring_latency_ms, cache_hit, tier_assigned, org_id

Provide: complete app structure, all controllers, middleware, startup lifespan, 35 async pytest tests with anyio.

PROMPT 04 — SvelteKit Sales Dashboard with Supabase Realtime

You are a senior Svelte/SvelteKit engineer and UI designer building a real-time B2B sales intelligence dashboard.

TASK: Build the LeadForge AI SvelteKit sales dashboard using Svelte 5 Runes and Supabase Realtime.

TECH: SvelteKit 2, Svelte 5 (Runes: \$state, \$derived, \$effect), TypeScript 5.4, TanStack Query v5, Tailwind 3.4.
Supabase client: @supabase/supabase-js v2 – realtime subscriptions + REST queries.

PAGES + ROUTES:

/dashboard – PipelineBoard (Kanban HOT/WARM/COLD) + TierBreakdownDonut + TopHotLeads
/lead/[id] – LeadDetailPage: ScoreGauge + SHAPWaterfall + FeatureTable + ScoreHistoryChart
/analytics – FunnelChart + ConversionBySource + WeeklyTrend + ModelHealthCard
/tiers – TierRuleEditor: sliders for HOT/WARM thresholds; live preview of tier changes
/settings – API key management, webhook URLs for Salesforce + HubSpot, notification prefs

KEY COMPONENTS:

1. LeadCard: score gauge (SVG circle progress), tier badge (colour-coded pill), company, top-1 reason, action button
2. KanbanBoard: 3 columns, IntersectionObserver infinite scroll, drag lead → tier override via PATCH
3. SHAPWaterfall: SVG waterfall chart (positive=violet bars right, negative=red bars left) for top-10 features
4. ScoreGauge: SVG arc, colour-coded 0-100, animated on load (Svelte transition)
5. RealtimeProvider: \$effect subscribes to Supabase 'leads' channel → updates \$state leads array live

REALTIME REQUIREMENT: When API scores a lead and writes to Supabase, the LeadCard in SvelteKit browser must update score + tier within 500ms – zero page refresh. Implement with `supabase.channel('leads').on('postgres_changes', {event: 'UPDATE', schema: 'public', table: 'leads'}, handler).`

DESIGN TOKENS (CSS vars): --bg #06030F, --surface #0E0720, --accent-primary #7C3AED, --tier-hot #F43F5E, --tier-warm #FB923C, --tier-cold #22D3EE, --text-primary #FAF5FF, --text-secondary #8B7BA8.

TESTING: 20 Vitest tests (LeadCard render, tier badge colour, gauge animation, realtime mock event handling).
Provide: complete SvelteKit project structure, all components, +page.svelte files, and test suite.

PROMPT 05 — Plotly Dash ML Analytics Dashboard

You are a senior data scientist and Python dashboard engineer building an ML diagnostics dashboard.

TASK: Build a Plotly Dash 2.x analytics dashboard for LeadForge AI with 5 interactive panels.

TECH: Dash 2.17, Plotly 5.x, Dash Bootstrap Components 1.x, dash-ag-grid for data tables.

PANEL 1 – Score Distribution:

go.Histogram of all lead scores (0-100, bin_size=5)

Vertical lines at tier thresholds: HOT $\geq$ 70 (red dashed), WARM $\geq$ 40 (amber dashed)

Colour segments: red (70-100), amber (40-69), cyan (0-39)

PANEL 2 – ROC & PR Curves (model comparison):

dcc.Dropdown: select 1 or 2 MLflow model versions to compare

go.Scatter: ROC curve for each model; area shaded; AUC in legend

go.Scatter: Precision-Recall curve; area under PR highlighted

Baseline: random classifier diagonal

PANEL 3 – Calibration Reliability Diagram:

go.Scatter: fraction_of_positives vs mean_predicted_probability (10 bins) – dots + line

go.Scatter: perfect calibration diagonal (dashed)

go.Bar: count of leads in each bin (secondary y-axis)

Annotation: 'Brier Score: {value}' + 'ECE: {value}'

PANEL 4 – SHAP Feature Importance:

go.Bar (horizontal): mean(|SHAP|) per feature, sorted desc, top 20

Colour: positive SHAP contribution = violet, negative = coral

Hover: exact SHAP value + feature description

PANEL 5 – Tier Analytics:

go.Pie: HOT / WARM / COLD count breakdown

go.Bar: actual conversion rate per tier (bars) vs model predicted rate (line) – grouped

go.Scatter: 90-day trend of HOT lead count with 7-day rolling average

INTERACTIVITY: Single dcc.Dropdown for model version selection → all 5 panels update via @callback.

STYLING: dbc.themes.CYBORG (dark); custom CSS: primary colour #7C3AED; card border #1E1040.

DEPLOYMENT: dash run server --host 0.0.0.0 --port 8050 in Docker; mounted at /ml-analytics/ in Nginx.

Provide: complete app.py, callbacks.py, data_loader.py with MLflow queries, and Dockerfile.

PROMPT 06 — CRM Webhook Integration (Salesforce + HubSpot)

You are a senior integration engineer building production CRM webhook integrations for a ML scoring API.

TASK: Build webhook handlers in Litestar that receive events from Salesforce and HubSpot, extract lead features, trigger scoring, and write results back to the CRM.

SALESFORCE INTEGRATION:

- Incoming: Salesforce Outbound Message (SOAP XML) on Lead.create / Lead.update
- Parse SOAP XML → extract fields: FirstName, LastName, Company, Title, Industry, AnnualRevenue, NumberOfEmployees, LeadSource, Email, Phone, ConvertedDate (null = not converted)
- Field mapping: Company → company_name; NumberOfEmployees → company_size_bucket (SMB/Mid/Ent)
- Authenticate: verify Salesforce org fingerprint via session_id header
- Write back: use simple_salesforce async client to PATCH Lead.LeadForge_Score__c + LeadForge_Tier__c fields

HUBSPOT INTEGRATION:

- Incoming: HubSpot webhook (JSON) on contact.creation / contact.propertyChange
- Verify HMAC signature: sha256(client_secret + request_body).hex == X-HubSpot-Signature-v3
- Extract: company, jobtitle, hs_email_opens_count, hs_email_last_send_date, num_contacted_notes
- Write back: PATCH /crm/v3/objects/contacts/{id} with leadforge_score + leadforge_tier properties

SHARED REQUIREMENTS FOR BOTH:

- POST /api/v1/webhooks/salesforce and POST /api/v1/webhooks/hubspot endpoints in Litestar
- After extraction: call await feature_store.upsert(lead_id, features) → Redis + Supabase
- Enqueue RQ job: score_lead.delay(lead_id, source='salesforce'|'hubspot')
- Idempotency: store webhook event_id in Supabase; skip if already processed
- Retry: if scoring fails, RQ retries 3× with exponential backoff
- Audit log: write every webhook receipt + scoring result to webhook_events table

Provide: complete Litestar controller, CRM client classes, idempotency middleware, 20 pytest tests with mock CRM responses.

PROMPT 07 — ML Model Monitoring & Drift Detection

You are a senior MLOps engineer responsible for keeping a production lead scoring model healthy over time.

TASK: Build a complete model monitoring and drift detection system for LeadForge AI.

MONITORING COMPONENTS:

1. FEATURE DRIFT (Alibi-Detect MMD):

- Weekly batch job: compare last 7 days incoming features vs training feature distribution
- Use KernelMMD test (`alibi_detect.cd.MMDDrift`); `kernel=GaussianRBF`; `p-value threshold=0.05`
- Run per feature group: company features, engagement features, timing features
- Output: `DriftReport(features_drifted: List[str], p_values: dict, severity: 'low'|'medium'|'high')`
- If `severity='high'`: fire Slack alert via webhook POST to #ml-alerts channel

2. PREDICTION DRIFT:

- Monitor score distribution shift: KS test (`scipy.stats.ks_2samp`) on daily score histograms
- Alert if HOT% drops > 5 percentage points vs 30-day baseline
- Alert if mean calibrated probability shifts > 0.05 vs training calibration baseline

3. PERFORMANCE DEGRADATION:

- Delay-labelled evaluation: leads scored 90 days ago → join with actual conversion outcome
- Compute rolling AUC-ROC, Brier Score, Precision@Tier for last 30 / 60 / 90 day windows
- Alert if AUC-ROC drops > 0.03 vs registered model AUC

4. AUTOMATIC RETRAINING TRIGGER:

- Conditions: `drift severity='high'` OR `AUC drops > 0.03` OR `Brier > 0.15`
- RQ job: `retrain_model.delay(trigger_reason, timestamp)`
- Retrain with last 90 days of data + original training set (sliding window)
- If new model AUC $\geq$ current model AUC - 0.01: auto-promote in MLflow registry
- If promotion: hot-reload model in Litestar app.state (zero-downtime swap)

5. MONITORING DASHBOARD (Dash Panel 6):

- 30-day AUC trend line + alert threshold line
- Per-feature drift bar chart (p-value bars, red if < 0.05)
- Retraining history timeline

Output: `monitoring.py`, `drift_detector.py`, `retraining_pipeline.py`, `rq_tasks.py`, and 20 `pytest` tests.

11 · EVALUATION & BUSINESS METRICS FRAMEWORK

Proving model quality to data scientists and conversion impact to business stakeholders

ML Model Metrics

METRIC	FORMULA / METHOD	BASELINE	TARGET	STRETCH
AUC-ROC	Area under ROC curve (threshold-agnostic)	0.74	0.92	0.95
AUC-PR	Area under Precision-Recall curve	0.31	0.58	0.67
Brier Score	Mean squared error of probabilities	0.19	0.12	0.09
ECE	Expected Calibration Error (10-bin)	0.09	0.03	0.018
F1 Score	Harmonic mean P+R at optimal threshold	0.52	0.68	0.75
Precision@HOT	HOT leads that actually convert	12%	35%	48%
Recall@HOT	Converters correctly labelled HOT	42%	71%	82%
SHAP Stability	Correlation of SHAP ranks across 3 seeds	0.81	0.95	0.98
Tier Consistency	Same tier on 2 identical leads	100%	100%	100%

Business Impact Metrics (Post-Deployment)

METRIC	MEASUREMENT METHOD	BASELINE	TARGET
Conversion rate — HOT tier	Won deals / HOT leads (30-day cohort)	12%	35%+
Conversion rate — WARM tier	Won deals / WARM leads (30-day cohort)	4%	12%+
Conversion rate — COLD tier	Won deals / COLD leads (30-day cohort)	1%	2%
Rep time on HOT leads	Time tracking — hours on HOT vs COLD	28%	65%+
Pipeline velocity	Avg days from lead to deal close	84 days	< 60 days
Forecast accuracy	Predicted conversions vs actual (monthly)	±35%	±10%
Sales quota attainment	% reps hitting quota (30-day post-launch)	62%	75%+

API Performance SLAs

OPERATION	P50	P95	P99	ERROR%
POST /score — cache hot features	45ms	120ms	200ms	< 0.1%
POST /score — cold path (PG)	80ms	180ms	300ms	< 0.5%
SHAP computation per lead	8ms	18ms	30ms	< 0.1%
GET /leads/ (paginated 20)	12ms	30ms	50ms	< 0.1%
POST /webhooks/crm (async RQ)	5ms	12ms	25ms	< 0.1%
Supabase Realtime broadcast	—	—	< 500ms	—
RQ batch: score 50k leads	—	—	< 5min	—

12 · REST API SPECIFICATION (LITESTAR)

18 production endpoints — msgspec DTOs — OpenAPI 3.0 — async throughout

METHOD	ENDPOINT	AUTH	RATE LIMIT	DESCRIPTION
POST	/api/v1/leads/{id}/score	JWT	500/min	Real-time score single lead; return score+tier+reasons
POST	/api/v1/leads/batch-score	Admin	10/min	Enqueue RQ batch job; return job_id
GET	/api/v1/leads/	JWT	200/min	Cursor-paginated leads; filter by tier, score range
POST	/api/v1/leads/	JWT	200/min	Create/upsert lead; auto-trigger async score
GET	/api/v1/leads/{id}/	JWT	500/min	Lead detail + latest score + SHAP reasons
PATCH	/api/v1/leads/{id}/	JWT	100/min	Update lead attributes; invalidate cache; re-score
GET	/api/v1/leads/{id}/score-history	JWT	200/min	Score trend last 90 days (timestamp, score, tier)
PATCH	/api/v1/leads/{id}/tier-override	Manager	50/min	Manual tier override with reason; audit logged
POST	/api/v1/webhooks/salesforce	HMAC	unlimited	Salesforce outbound message → async score trigger
POST	/api/v1/webhooks/hubspot	HMAC	unlimited	HubSpot webhook → async score trigger
GET	/api/v1/analytics/pipeline	JWT	60/min	HOT/WARM/COLD counts + conversion rates + trend
GET	/api/v1/analytics/model	JWT	30/min	Current AUC, Brier, ECE, top-5 features from MLflow
GET	/api/v1/tiers/	Admin	30/min	Get current tier rule configuration
PUT	/api/v1/tiers/	Admin	5/min	Update tier thresholds; re-tier all active leads
GET	/api/v1/jobs/{job_id}	JWT	100/min	RQ job status (pending/started/finished/failed)
GET	/api/v1/events/	Admin	30/min	Audit log of all score events + webhook receipts
GET	/api/v1/health/	—	unlimited	Health check: DB, Redis, RQ, model loaded status
GET	/schema	—	unlimited	OpenAPI 3.0 JSON schema; Swagger UI at /schema/swagger

Score Response Schema

```
POST /api/v1/leads/LEAD-00421/score → HTTP 200
{
  "lead_id": "LEAD-00421",
  "score": 83,
  "calibrated_probability": 0.831,
  "tier": "HOT",
  "tier_display": "■ HOT",
  "sla": "Call within 24 hours",
  "recommended_action": "Assign AE; create Salesforce task",
  "short_reason": "Enterprise size & demo request drive high score",
  "reasons": [
    {"feature": "company_size", "value": "Enterprise", "shap": 0.18, "text": "Enterprise company – significantly increases conversion likelihood"},
    {"feature": "demo_requested", "value": true, "shap": 0.14, "text": "Demo requested – notably positive: converts 3× baseline"},
    {"feature": "days_since_last_contact", "value": 45, "shap": -0.11, "text": "45 days since contact – notably reduces score: re-engage urgently"}
  ],
}
```

```
"scored_at": "2024-11-15T09:23:41Z",  
"model_version": "LeadForge-XGBoost-v3",  
"latency_ms": 87,  
"cache_hit": false  
}
```

13 · DEVOPS — RAILWAY + DOCKER + CI/CD

Production deployment — Railway for backend, Cloudflare Pages for SvelteKit frontend

Railway Production Services

SERVICE	RAILWAY CONFIG	DOCKER IMAGE	RESOURCES	SCALE
litestar_api	railway.toml service 1	python:3.12-slim	1 vCPU / 2GB	1-5 replicas; CPU>70%
rq_worker	railway.toml service 2	python:3.12-slim	2 vCPU / 4GB	1-3; queue depth>50
plotly_dash	railway.toml service 3	python:3.12-slim	1 vCPU / 1GB	Fixed 1 replica
redis	Railway Redis plugin	Managed	512MB	Managed; auto-failover

Cloudflare Pages — SvelteKit Frontend

STEP	ACTION	TOOL
1	Connect GitHub repo to Cloudflare Pages project	Cloudflare dashboard
2	Build command: npm run build; output: .svelte-kit/cloudflare	SvelteKit adapter-cloudflare
3	Environment vars: PUBLIC_SUPABASE_URL, PUBLIC_SUPABASE_ANON_KEY, MLFLOW_TRACKING_URI	Cloudflare env settings
4	Custom domain: leadforgeai.com → Cloudflare DNS → Pages	CF DNS management
5	Preview deployments: every PR gets a unique preview URL	CF Pages auto
6	Edge SSR: Svelte routes run at CF edge (200+ global locations)	adapter-cloudflare

CI/CD Pipeline — GitHub Actions

STAGE	TRIGGER	ACTIONS	PASS GATE
Lint	Every push	ruff check, mypy --strict, svelte-check, tsc	0 errors
Test	Every PR	pytest 60 tests + vitest 20 tests (parallel)	100% pass
Security	Every PR	pip-audit, npm audit, trivy image scan	0 critical
Build	Merge to main	docker build + push to GitHub Container Registry	< 5min build
Staging	Auto	railway up --environment staging; smoke tests	200 on /health/
Approve	Manual	GitHub env protection — 1 reviewer required	Approval received
Prod	Post-approve	railway up --environment production; verify SLAs	p95 < 200ms
Rollback	Auto trigger	railway rollback to previous deploy if error > 1%	Triggered if breach

Local Development — Zero to Running in 4 Minutes

```
git clone https://github.com/yourorg/leadforge-ai && cd leadforge-ai
cp .env.example .env # Add SUPABASE_URL, SUPABASE_KEY, MLFLOW_URI
supabase start # Local Supabase on :54321
supabase db push # Run all migrations
docker compose up --build # Litestar :8000, Dash :8050, Redis, RQ
cd frontend && npm install && npm run dev # SvelteKit on :5173
```

```
# Access:
# SvelteKit Dashboard: http://localhost:5173
# Litestar API: http://localhost:8000
# OpenAPI Swagger: http://localhost:8000/schema/swagger
# Plotly Dash: http://localhost:8050
# MLflow UI: http://localhost:5000
# RQ Dashboard: http://localhost:9181
# Supabase Studio: http://localhost:54323
```

14 · RISK REGISTER & SALES STRATEGY

Technical risks, business risks, and how to win every client demo

RISK	LIKELIHOOD	IMPACT	MITIGATION
Severe class imbalance (7% conversion)	HIGH	HIGH	SMOTE + scale_pos_weight + class_weight='balanced'; report PR-AUC not ROC
Calibration fails: XGB overconfident	HIGH	HIGH	CalibratedClassifierCV with cv=5; verify with reliability diagram before deployment
SHAP slow on large feature sets	MEDIUM	MEDIUM	TreeExplainer (fast for XGB); pre-compute SHAP on batch; < 10ms per lead
CRM data quality: missing key features	HIGH	MEDIUM	NullImputer for all fields; fallback tier rules if > 30% features null
Feature drift renders model stale after 6 months	HIGH	HIGH	MMD drift detector weekly; auto-retrain trigger; model card with retrain schedule
Client CRM uses non-standard field names	HIGH	MEDIUM	Field mapping config YAML per CRM (Salesforce, HubSpot, Pipedrive, etc)
Railway cold-start latency (first request)	MEDIUM	MEDIUM	Keep-alive health check pings every 5 min; Railway 'Always On' setting
Supabase Realtime WebSocket disconnects	LOW	MEDIUM	Reconnection logic in Svelte \$effect with exponential backoff
Sales team distrust of AI scores	HIGH	MEDIUM	SHAP reasons shown on every lead; tier override allowed; human always in loop
GDPR: lead data is PII	MEDIUM	HIGH	Supabase RLS isolates orgs; data deletion endpoint; DPA template provided
Optuna tuning GPU cost overrun	LOW	LOW	CPU-only XGBoost tuning; 200 trials < 4h on 4-core; no GPU needed for tuning

How to Win the Client Demo in 8 Minutes

- Minute 1-2: Show the Pipeline Board — HOT leads glowing red, WARM in amber. Ask: 'Does your team know which leads to call first?' Pause.
- Minute 3: Click a HOT lead. Show the SHAP waterfall. Say: 'This is why we scored it 83 — Enterprise + demo request + 12 email opens. Not a black box.'
- Minute 4: Show the calibration curve in Plotly Dash. Say: 'When we say 80%, 79.3% of those leads convert. You can build your revenue forecast on this.'
- Minute 5: Live demo — POST a lead via Swagger UI, show it appear in the Pipeline Board in < 500ms. Supabase Realtime magic.
- Minute 6: Show the Tier Rules editor. Say: 'You can change the HOT threshold from 70 to 75 — all leads re-tier instantly. You own the model's behaviour.'
- Minute 7: Open the RQ Dashboard — show 50k leads scored in 4 minutes. Say: 'Your entire CRM, scored overnight, automatically.'
- Minute 8: Show the drift alert Slack message. Say: 'If your customer profile changes, we know before you do — and the model retrains itself.'

15 · TIMELINE, MILESTONES & PRICING

16-week delivery plan, exit gates, and what to charge for this system

16-Week Milestone Roadmap

WK	PHASE	MILESTONE	EXIT GATE
1	Phase 1	Monorepo + Docker + Supabase schema	8 services healthy; RLS tested
2	Phase 1	50k synthetic leads generated + validated	GE: 30/30 expectations pass
3	Phase 1	Feature pipeline + DVC + MLflow setup	dvc repro → clean parquet; MLflow UI
4	Phase 2	XGBoost baseline (no tuning) trained	AUC-ROC > 0.80 baseline logged
5	Phase 2	Optuna 200-trial tuning complete	Best params saved; 200 trials in MLflow
6	Phase 2	Calibrated model + SHAP + TierEngine	Brier ≤ 0.12; SHAP reasons correct
7	Phase 2	Plotly Dash dashboard all 5 panels	All panels with real model data
8	Phase 3	Litestar API all 18 endpoints live	Swagger complete; 35 tests green
9	Phase 3	SvelteKit Pipeline Board + LeadCard	HOT/WARM/COLD board renders 50k leads
10	Phase 3	SHAP Waterfall + Score History in SvelteKit	Lead detail page fully functional
11	Phase 3	Supabase Realtime end-to-end	Score → DB → browser in < 500ms
12	Phase 3	CRM webhooks (Salesforce + HubSpot)	Webhook → async score → CRM write-back
13	Phase 4	Railway production deploy (4 services)	All Railway services healthy at prod URL
14	Phase 4	Cloudflare Pages SvelteKit deploy	CF Pages live; LCP < 2.5s
15	Phase 4	Load test + drift monitoring live	300 concurrent; p95 < 200ms; drift alerts
16	Phase 4	Demo video + client docs delivered	8-min Loom; 4 markdown guides; Postman

Freelance & Consulting Pricing Guide

DELIVERABLE	SCOPE	PRICE	TIMELINE
Full system (all 4 phases)	Complete blueprint as-built	\$18,000–\$45,000	16 weeks
MVP: API + Model + Scoring only	Litestar API + XGBoost + calibration	\$6,000–\$12,000	7 weeks
CRM Integration addon	Salesforce or HubSpot webhook	\$3,000–\$6,000	2 weeks
SvelteKit Dashboard only	Frontend only (API pre-built)	\$4,000–\$8,000	4 weeks
Plotly Dash Analytics only	5-panel ML dashboard	\$2,500–\$5,000	2 weeks
Custom model training on client CRM	Client data + retrain + handoff	\$4,000–\$9,000	3 weeks
RevOps consulting + training	Model + team training sessions	\$8,000–\$20,000	4 weeks
Enterprise white-label license	Full codebase + 1yr support	\$60,000–\$150,000	16+4 weeks
Monthly SaaS retainer	Hosting + model refresh + support	\$1,200–\$3,500/mo	Ongoing

